

Views & Materialized Views

Module 5 • Build a clean, secure and reusable reporting layer.

MODULE

05

60 minutes

15 min Explain
10 min Demonstrate
25 min Hands-on
10 min Review

developer_lab continues

Module 5 turns queries into a reusable data layer

1

SIMPLIFY

Hide joins and calculations behind a stable, meaningful name.

2

CONTROL

Expose only the rows and columns that a consumer needs.

3

ACCELERATE

Store expensive query results and refresh them deliberately.

A view stores a query definition; a materialized view stores its result.

By the end, learners can complete five tasks

01

Create and query a standard view

02

Use a view as a permission boundary

03

Recognize when a view is automatically updatable

04

Create, refresh and index a materialized view

05

Choose the right object for freshness and speed

SUCCESS = BUILD THE REPORTING LAYER

A standard view runs its query when you use it

1 DEFINITION

PostgreSQL stores the SELECT statement—not a copy of its rows.

2 EXECUTION

A query against the view is planned with its underlying relations.

3 RESULT

Current committed base-table data is returned.

A view improves abstraction and governance—not automatically performance.

Create a view for order totals

```
CREATE OR REPLACE VIEW app.order_summary AS
SELECT o.order_id, o.customer_id, o.status,
       o.created_at,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0)
       AS order_total
FROM app.orders AS o
LEFT JOIN app.order_items AS oi USING (order_id)
GROUP BY o.order_id;
```

WHY THIS VIEW?

- One row per order
- Reusable total calculation
- Stable column names

CHECK

```
SELECT *
FROM app.order_summary
ORDER BY order_id;
```

Query a view like a table—and keep filtering

```
SELECT order_id, status, order_total  
FROM app.order_summary  
WHERE status = 'PAID'  
      AND order_total >= 500  
ORDER BY order_total DESC;
```

PREDICATES

PostgreSQL can push useful filters into the combined plan.

INDEXES

Indexes on the underlying tables remain important.

Use EXPLAIN on the outer query to inspect the full execution plan.

Views separate a public contract from internal tables

BASE TABLES

- Operational columns
- Internal identifiers
- Write-oriented structure
- Sensitive attributes

VIEW CONTRACT

- Named business fields
- Selected rows and columns
- Reusable calculations
- Consumer-friendly grain

CONSUMERS

- Dashboards
- Applications
- Analysts
- External integrations

Treat view columns as an interface: rename or remove them carefully.

Grant access to the view—not every source table

```
CREATE ROLE report_reader;  
  
GRANT USAGE ON SCHEMA app TO report_reader;  
GRANT SELECT ON app.order_summary TO report_reader;  
  
REVOKE ALL ON app.orders, app.order_items  
FROM report_reader;
```

LEAST PRIVILEGE

Expose only objects required for the reporting task.

TEST THE ROLE

```
SET ROLE report_reader;  
SELECT * FROM app.order_summary;  
RESET ROLE;
```

Permissions and ownership need deliberate testing; a view is not magic security.

Simple views can be automatically updatable

USUALLY UPDATABLE

- One base table
- Direct column references
- Row filtering
- No grouping or set operation

USUALLY READ-ONLY

- Aggregates or GROUP BY
- DISTINCT
- UNION / INTERSECT / EXCEPT
- Many complex joins

When in doubt, test INSERT, UPDATE and DELETE inside a transaction you can roll back.

WITH CHECK OPTION keeps writes inside the view rule

```
CREATE VIEW app.open_orders AS  
SELECT order_id, customer_id, status, created_at  
FROM app.orders  
WHERE status IN ('NEW', 'PAID')  
WITH LOCAL CHECK OPTION;  
  
UPDATE app.open_orders  
SET status = 'CANCELLED'  
WHERE order_id = 1; -- rejected
```

WITHOUT CHECK

A row can be updated so it no longer appears through the view.

WITH CHECK

PostgreSQL rejects a write that violates the view predicate.

Dependencies make schema changes visible—and intentional

```
\d+ app.order_summary  
  
SELECT pg_get_viewdef('app.order_summary'::regclass, true);  
  
DROP VIEW app.order_summary;  
-- fails if dependent objects exist unless handled
```

CHANGE SAFELY

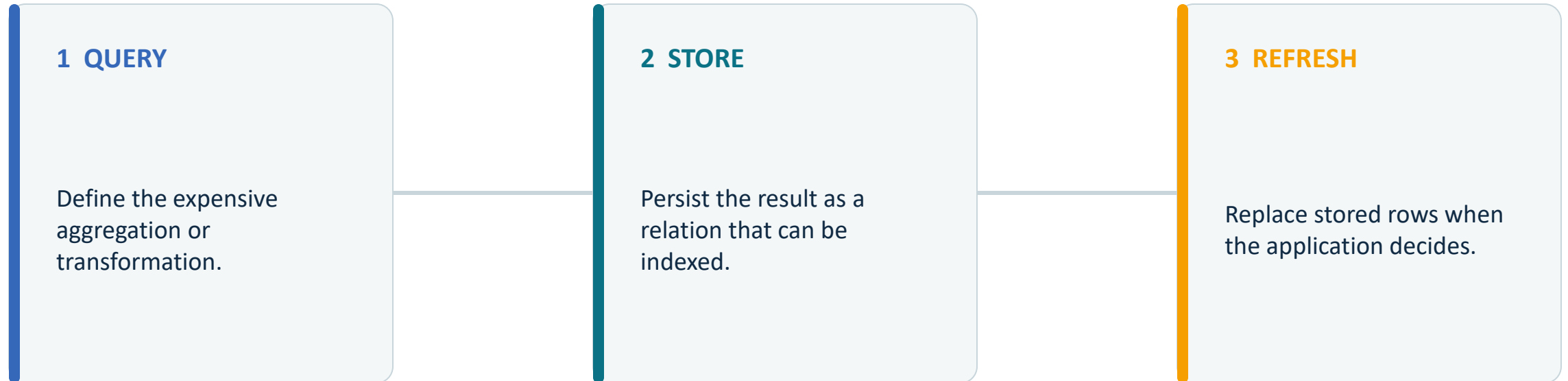
Version the view contract and test dependent applications.

AVOID CASCADE BY HABIT

DROP ... CASCADE can remove more objects than intended.

CREATE OR REPLACE VIEW cannot arbitrarily change or remove existing output columns.

A materialized view stores rows at refresh time



Fast reads come with a freshness gap and refresh work.

Choose based on freshness, cost and write behavior

OBJECT	FRESHNESS	READ COST	WRITES
STANDARD VIEW	Current at query time	Runs underlying query	May be updatable
MATERIALIZED VIEW	Current at last refresh	Reads stored result	Read-only result

Materialize only when repeated read savings justify refresh complexity.

Create a daily sales snapshot

```
CREATE MATERIALIZED VIEW app.daily_sales_mv AS
SELECT o.created_at::date AS sales_date,
       COUNT(DISTINCT o.order_id) AS order_count,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0)
       AS sales_total
FROM app.orders AS o
JOIN app.order_items AS oi USING (order_id)
WHERE o.status = 'PAID'
GROUP BY o.created_at::date
WITH DATA;
```

WITH DATA

Runs the query immediately and stores its current result.

WITH NO DATA

Creates an unscannable object until the first successful refresh.

Refresh replaces the stored result

```
REFRESH MATERIALIZED VIEW app.daily_sales_mv;  
  
-- after a qualifying unique index exists  
REFRESH MATERIALIZED VIEW CONCURRENTLY  
app.daily_sales_mv;
```

REGULAR REFRESH

Simpler; blocks readers while replacement is performed.

CONCURRENT REFRESH

Allows reads, takes longer and has specific prerequisites.

CONCURRENTLY requires a qualifying **UNIQUE** index and cannot populate an unscannable view.

Index the stored result for its consumer queries

```
CREATE UNIQUE INDEX daily_sales_mv_date_uq  
ON app.daily_sales_mv (sales_date);
```

```
CREATE INDEX daily_sales_mv_total_idx  
ON app.daily_sales_mv (sales_total DESC);
```

```
ANALYZE app.daily_sales_mv;
```

UNIQUE INDEX

Supports row matching needed by concurrent refresh.

QUERY INDEX

Match frequent filters or ordering on the materialized result.

Indexes on base tables do not automatically become indexes on the materialized view.

Demo: prove the freshness gap

```
SELECT * FROM app.daily_sales_mv  
ORDER BY sales_date DESC;
```

-- insert or change a PAID order in base tables

```
SELECT * FROM app.daily_sales_mv; -- unchanged  
REFRESH MATERIALIZED VIEW app.daily_sales_mv;  
SELECT * FROM app.daily_sales_mv; -- updated
```

ASK FIRST

Will the snapshot change immediately after a base-table write?

OBSERVE

Base-table truth and stored snapshot differ until REFRESH completes.

Hands-on: build the reporting layer

01

Create app.order_summary

02

Query paid orders above a chosen total

03

Create app.open_orders with CHECK OPTION

04

Create app.daily_sales_mv

05

Add the unique index and refresh the snapshot

WORK IN developer_lab

Lab validation: five checks must pass

```
\dv app.*  
\dm app.*
```

```
SELECT count(*) FROM app.order_summary;  
SELECT count(*) FROM app.daily_sales_mv;
```

```
SELECT indexname  
FROM pg_indexes  
WHERE schemaname = 'app'  
AND tablename = 'daily_sales_mv';
```

EXPECTED EVIDENCE

order_summary exists
open_orders exists
daily_sales_mv exists
Counts return without errors
Unique date index is listed

Troubleshoot by matching symptom to cause

SYMPTOM	LIKELY CAUSE	ACTION
cannot change name of view column	Output contract changed	Keep names or recreate deliberately
cannot update view	View is not automatically updatable	Write base table or use a trigger
materialized view is unpopulated	Created WITH NO DATA	Run REFRESH MATERIALIZED VIEW
CONCURRENTLY prerequisite error	No qualifying UNIQUE index	Create the required unique index

Read the complete PostgreSQL error before changing the object.

Knowledge check: explain the choice

SCENARIO A

A dashboard must show committed order changes immediately.

Which object—and why?

SCENARIO B

A costly daily aggregation is read thousands of times.

Which object—and why?

SCENARIO C

A reporting role must not read customer contact details.

How would you grant access?

Suggested: standard view • materialized view • SELECT on a restricted view only.

Module 5 takeaway: separate contract, freshness and speed

1

VIEW

Reusable query and stable interface over current base-table data.

2

MATERIALIZE

Stored result for repeated reads, with an explicit freshness policy.

3

OPERATE

Grant narrowly, index for consumers and monitor refresh duration.

Next: Module 6 — PL/pgSQL functions and exception handling.